

```

-- =====
-- INÍCIO ATIVIDADE 16
-- =====

-- =====
-- PREPARATIVOS
-- =====
-- Cria uma coluna na tabela livro para armazenar a quantidade de livros
ALTER TABLE livro
ADD COLUMN quantidade INTEGER DEFAULT 5;

-- Cria tabela de log
CREATE TABLE log_livro (
    id_log SERIAL PRIMARY KEY,
    titulo VARCHAR(100),
    quantidade_antiga INTEGER,
    quantidade_nova INTEGER,
    data_log TIMESTAMP
);

-- Cria tabela de log de exclusao
CREATE TABLE log_exclusao_livro (
    id_log SERIAL PRIMARY KEY,
    titulo VARCHAR(100),
    data_log TIMESTAMP,
    mensagem VARCHAR(100)
);

-- =====
-- Exercício 1 - Trigger para Bloqueio de Exclusão
-- =====
-- A coordenação deseja impedir que livros sejam removidos do sistema caso
-- ainda existam exemplares disponíveis.
-- Desenvolva:
-- a)
-- Uma FUNCTION chamada:
-- bloquear_exclusao()
-- que:
-- - impeça DELETE quando:
--     quantidade > 0
-- - utilize:
--     RAISE EXCEPTION
-- para exibir mensagem apropriada.
-- b)
-- Uma TRIGGER chamada:
-- - trg_bloquear_exclusao
-- executada BEFORE DELETE na tabela:
--     livro
-- =====
CREATE OR REPLACE
FUNCTION
bloquear_exclusao()
RETURNS TRIGGER
AS $$
BEGIN
    IF OLD.quantidade > 0 THEN
        RAISE EXCEPTION 'Não podemos excluir registros enquanto há livros
                        existentes!';
    END IF;

    RETURN OLD;
END;

$$ LANGUAGE plpgsql;

-- =====
-- Trigger
-- =====
CREATE TRIGGER trg_bloquear_exclusao
BEFORE
DELETE

```

```

        ON
        livro
FOR EACH ROW
EXECUTE FUNCTION bloquear_exclusao();
-- =====
-- TESTE
-- =====
DELETE FROM livro WHERE id_livro = 1; -- temos 5 livros
INSERT INTO livro VALUES (7,'O Alienista',1882,2,2,112,0); -- quantidade 0
DELETE FROM livro WHERE id_livro = 7; -- deve apagar o livro "O Alienista"

-- =====
-- Exercício 2 - Registro Automático de Exclusões
-- =====
-- A biblioteca deseja manter histórico de livros removidos do sistema.
-- Desenvolva:
-- a) Uma FUNCTION chamada:
--     log_exclusao_livro()
-- que registre:
--     - título do livro removido;
--     - data e hora da exclusão;
--     - mensagem informativa no log.
-- b) Uma TRIGGER chamada:
--     trg_log_exclusao
-- executada AFTER DELETE na tabela:
--     livro
-- =====
CREATE OR REPLACE
FUNCTION
log_exclusao_livro()
RETURNS TRIGGER
AS $$
BEGIN
    INSERT
    INTO
    log_exclusao_livro(
        titulo,
        data_log,
        mensagem)
VALUES (
    OLD.titulo,
    CURRENT_TIMESTAMP,
    'Registro removido.');
```

```

RETURN OLD;
END
$$ LANGUAGE plpgsql;
-- =====
-- Trigger
-- =====
CREATE TRIGGER trg_log_exclusao_livro
AFTER
DELETE
    ON
    livro
FOR EACH ROW
EXECUTE FUNCTION log_exclusao_livro();
-- =====
-- TESTE
-- =====
DELETE FROM livro WHERE id_livro = 1; -- temos 5 livros
INSERT INTO livro VALUES (8,'O Alienista',1882,2,2,112,0); -- quantidade 0
DELETE FROM livro WHERE id_livro = 8; -- deve apagar o livro e registrar no log

-- =====
-- Exercício 3 - Controle de Aumento Excessivo de Estoque
-- =====
-- Durante auditorias, foi identificado que alguns operadores cadastraram quan-
-- tidades irreais de livros.
-- Crie:
```

```

-- a) Uma FUNCTION chamada:
--     validar_limite_estoque() que:
--         -impeça UPDATE quando:
--             -quantidade > 100
--         -apresente mensagem de erro usando:
--             RAISE EXCEPTION
-- b) Uma TRIGGER chamada:
--     trg_validar_limite
--         executada BEFORE UPDATE na tabela livro
-- =====
CREATE OR REPLACE
FUNCTION
validar_limite_estoque()
RETURNS TRIGGER
AS $$
BEGIN
    IF NEW.quantidade >100 THEN
        RAISE EXCEPTION 'Solicite autorização para inserir mais de 100
        quantidades.';
    END IF;

    RETURN NEW;
END;

$$ LANGUAGE plpgsql;
-- =====
-- Trigger
-- =====
CREATE TRIGGER trg_validar_limite_estoque
BEFORE
UPDATE

    ON
        livro
FOR EACH ROW
EXECUTE FUNCTION validar_limite_estoque();
-- =====
-- TESTE
-- =====
UPDATE livro SET quantidade = 100 WHERE id_livro = 1; --aumenta de 5 para 100!
UPDATE livro SET quantidade = 101 WHERE id_livro = 1; --impede a atualização.

-- =====
-- Exercício 4 - Análise de Trigger BEFORE e AFTER
-- =====
-- Observe o cenário:
-- A equipe criou uma trigger:
--     BEFORE UPDATE
-- para validar estoque e outra:
--     AFTER UPDATE
-- para registrar logs.
-- Explique detalhadamente:
-- a) A diferença entre triggers BEFORE e AFTER.
-- b) Qual delas deve ser usada para validação.
-- c) Qual delas deve ser usada para auditoria.
-- d) Por que a ordem de execução é importante em bancos de dados reais.
-- =====

-- =====
-- RESPOSTAS
-- =====
-- a) BEFORE executa a ação antes da transação ocorrer; AFTER executa a ação
-- após sua conclusão.
-- b) Para validação, deve ser usada BEFORE.
-- c) Para auditoria, deve ser usada AFTER, já que o fato já ocorreu.
-- d) Pois a ordem importa na manutenção da integridade dos dados lá existentes.
-- Além da integridade, a ordem é crucial para a performance e o controle de
-- erros. Se tivermos várias triggers do mesmo tipo (todas 'BEFORE', por
-- exemplo), a ordem em que elas são criadas define a ordem de execução. Supondo
-- que uma trigger de validação de estoque falhar, ela impede que todas as
-- triggers subsequentes sejam executadas, o que evita criar falsos logs.

```

```

-- =====
-- Exercício 5 - Reflexão sobre Integridade e Automação
-- =====
-- Uma empresa decidiu remover todas as triggers do banco e deixar as
-- validações apenas na aplicação.
-- Analise criticamente:
-- a) Quais riscos essa decisão pode gerar.
-- b) Quais vantagens existem em manter regras diretamente no banco.
-- c) Explique como triggers ajudam na integridade e consistência dos dados.
-- d) Cite um exemplo real de uso de triggers em sistemas corporativos.
-- =====

-- =====
-- RESPOSTAS
-- =====
-- a)A remoção total das triggers e a dependência exclusiva da aplicação para
-- validações trazem riscos críticos:
-- -Perda de regras em acessos diretos: Se um administrador ou desenvolvedor
-- executar um comando SQL manualmente direto no banco, as regras de negó-
-- cio da aplicação não serão aplicadas, o que pode corromper os dados.
-- -Vulnerabilidade a falhas na aplicação: Se o frontend falhar ou o backend
-- apresentar um bug na persistência, as regras críticas deixam de existir,
-- pois não há uma última defesa no banco de dados.
-- -Inconsistência entre múltiplos sistemas: Caso a empresa utilize diferentes
-- sistemas (ex: um app mobile e um sistema web) acessando o mesmo banco, a
-- lógica de validação teria que ser duplicada em todos eles, aumentando o
-- risco de discrepâncias.

-- b)Manter certas regras no banco de dados oferece benefícios estratégicos:
-- -Centralização: A regra fica em um único lugar, garantindo que qualquer
-- sistema que acesse o banco siga o mesmo padrão.
-- -Segurança e Integridade: O banco protege a si mesmo contra dados inválidos,
-- independentemente de onde venha a requisição.
-- -Auditoria Automática: É possível gerar históricos de alterações (logs) de
-- forma nativa e automática, sem depender da lógica da aplicação.
-- -Padronização: Garante que tarefas manuais e repetitivas sejam automatiza-
-- das, eliminando o erro humano no processo de atualização de dados
-- relacionados.

-- c)Como triggers ajudam na integridade e consistência
-- -As triggers funcionam como sensores automáticos que reagem a eventos
-- (INSERT, UPDATE, DELETE) para garantir que o estado do banco permaneça
-- consistente:
-- -Validação Preventiva (BEFORE): Podem bloquear a inserção de dados inváli-
-- dos, como impedir que um livro seja cadastrado com número de páginas negativo.
-- -Sincronização Automática (AFTER): Garantem que, ao realizar uma ação
-- (como um empréstimo), outras tabelas sejam atualizadas instantaneamente
-- (como a redução do estoque), mantendo o equilíbrio dos dados.
-- -Garantia de Consistência: Elas operam dentro do conceito de transações,
-- assegurando que, se a trigger falhar, a operação principal também falhe,
-- evitando dados quebrados ou incompletos.

-- d)Exemplo real de uso em sistemas corporativos
-- -E-commerce: Atualização automática de estoque sempre que um pedido é fatu-
-- rado.
-- -Setor Bancário: Registro de toda e qualquer movimentação financeira em ta-
-- belas de log para fins de auditoria e segurança.
-- -RH: Registro de histórico de alterações salariais, armazenando o valor an-
-- tigo e o novo, além de quem fez a alteração e quando.

-- Conclusão: Embora o excesso de triggers possa dificultar o debug e impactar
-- a performance, removê-las completamente é perigoso. As boas práticas sugerem
-- que regras críticas de integridade devem permanecer no banco de dados, en-
-- quanto lógicas de negócio complexas e regras visuais devem ficar na apli-
-- cação.

-- =====
-- TÉRMINO ATIVIDADE 16
-- =====

```

General



<https://creativecommons.org/licenses/by-nc-sa/4.0/>

BRT	en	Writable	Smart Insert	63:21:1551	Sel: 0 0	:
-----	----	----------	--------------	------------	------------	---

FileEditNavigateSearchSQL EditorDatabaseWindowHelp

Connections x

Filter connections by name

postgres localhost:5432

Databases

Aula6

Aula7

PROVA_3

aula12

bdaula

bdaula_arley

bdteste

biblioteca

Schemas

public

Tables

aluno24K

autor24K

editora24K

emprestimo24K

emprestimo_livro24K

livro24K

Columns

Constraints

Foreign Keys

Indexes

Dependencies

References

Partitions

Triggers

trg_log_exclusao_livro

Rules

Policies

log_exclusao_livro24K

log_livro8K

Foreign Tables

Views

Materialized Views

Indexes

Functions

Sequences

Data types

*<postgres> schema.sql x

log_exclusao_livro

livro

-- - impeça DELETE quando:
-- quantidade > 0
-- - utilize:
-- RAISE EXCEPTION
-- para exibir mensagem apropriada.
-- b)
-- Uma TRIGGER chamada:
-- - trg_bloquear_exclusao
-- executada BEFORE DELETE na tabela:
-- livro
-- =====

CREATE OR REPLACE
FUNCTION
bloquear_exclusao()
RETURNS TRIGGER
AS \$\$
BEGIN
 IF OLD.quantidade >0 THEN
 RAISE EXCEPTION 'Não podemos excluir registros enquanto há livros existentes!';
 END IF;

 RETURN OLD;
END;

\$\$ LANGUAGE plpgsql;
-- =====
-- Trigger
-- =====

CREATE TRIGGER trg_bloquear_exclusao
BEFORE
DELETE
 ON
 livro
FOR EACH ROW
EXECUTE FUNCTION bloquear_exclusao();

Statistics 1 x

Name	Value
Updated Rows	0
Execute time	0.003s
Start time	Sun Jun 07 15:48:34 BRT 2026
Finish time	Sun Jun 07 15:48:34 BRT 2026
Query	-- ===== -- Trigger -- ===== CREATE TRIGGER trg_bloquear_exclusao BEFORE DELETE

General

BRTenWritableSmart Insert73 : 38 : 1725Sel: 0 | 0

43°CPU 9%MEM

87%15:48

FileEditNavigateSearchSQL EditorDatabaseWindowHelp

Connections x

Filter connections by name

emprestimo24K

emprestimo_livro24K

livro24K

Columns

123 id_livro (int4)

A-Z titulo (varchar(150))

123 ano_publicacao (int4)

123 id_autor (int4)

123 id_editora (int4)

123 num_paginas (int4)

123 quantidade (int4)

Constraints

Foreign Keys

Indexes

Dependencies

References

Partitions

Triggers

trg_bloquear_exclusao

Rules

Policies

Foreign Tables

Views

Materialized Views

Indexes

Functions

apagar_autor(int4)

atualizar_pg_livro(int4, int4)

info_autor(int4)

inserir_livro(varchar, int4, int4, int4, int4)

inserir_livro_com_verificacao(varchar, int4, int4, int4, int4)

f media_paginas_autor(int4)

Sequences

Data types

Aggregate functions

Event Triggers

Extensions

Storage

*<postgres> schema.sql x livro

-- RAISE EXCEPTION

-- para exibir mensagem apropriada.

-- b)

-- Uma TRIGGER chamada:

-- - trg_bloquear_exclusao

-- executada BEFORE DELETE na tabela:

-- livro

-- =====

CREATE OR REPLACE

FUNCTION

bloquear_exclusao()

RETURNS TRIGGER

AS \$\$

BEGIN

IF OLD.quantidade >0 THEN

RAISE EXCEPTION 'Não podemos excluir registros enquanto há livros existentes!';

END IF;

RETURN OLD.quantidade;

END;

\$\$ LANGUAGE plpgsql;

-- =====

-- Trigger

-- =====

CREATE TRIGGER trg_bloquear_exclusao

BEFORE DELETE ON livro

FOR EACH ROW

EXECUTE FUNCTION bloquear_exclusao();

-- =====

-- TESTE

-- =====

DELETE FROM livro WHERE id_livro = 1; -- temos 5 livros

INSERT INTO livro VALUES (7, '0 Alienista', 1882, 2, 2, 112, 0); -- quantidade 0

DELETE FROM livro WHERE id_livro = 7; -- deve apagar o livro "0 Alienista"

-- =====

Statistics 1 x

SQL Error [P0001]: ERROR: Não podemos excluir registros enquanto há livros existentes!

Where: PL/pgSQL function bloquear_exclusao() line 4 at RAISE

Go to Error

Show log

Details >>

-- =====

-- TESTE

-- =====

DELETE FROM livro WHERE id_livro = 1

General

BRTenWritableSmart Insert55 : 56 : 1430Sel: 0 | 0

41% CPU78%

MEM

87% 15:10

FileEditNavigateSearchSQL EditorDatabaseWindowHelp

SQLCommitRollbackAutopostgrespublic@biblioteca

Search

Connections

Filter connections by name

emprestimo24K

emprestimo_livro24K

livro24K

Columns

123 id_livro (int4)

AZ titulo (varchar(150))

123 ano_publicacao (int4)

123 id_autor (int4)

123 id_editora (int4)

123 num_paginas (int4)

123 quantidade (int4)

Constraints

Foreign Keys

Indexes

Dependencies

References

Partitions

Triggers

trg_bloquear_exclusao

Rules

Policies

Foreign Tables

Views

Materialized Views

Indexes

Functions

apagar_autor(int4)

atualizar_pg_livro(int4, int4)

info_autor(int4)

inserir_livro(varchar, int4, int4, int4, int4)

inserir_livro_com_verificacao(varchar, int4, int4, int4, int4)

f media_paginas_autor(int4)

Sequences

Data types

Aggregate functions

Event Triggers

Extensions

Storage

*<postgres> schema.sql x livro

-- RAISE EXCEPTION

-- para exibir mensagem apropriada.

-- b)

-- Uma TRIGGER chamada:

-- - trg_bloquear_exclusao

-- executada BEFORE DELETE na tabela:

-- livro

-- =====

CREATE OR REPLACE

FUNCTION

bloquear_exclusao()

RETURNS TRIGGER

AS \$\$

BEGIN

IF OLD.quantidade >0 THEN

RAISE EXCEPTION 'Não podemos excluir registros enquanto há livros existentes!';

END IF;

END;

\$\$ LANGUAGE plpgsql;

-- =====

-- Trigger

-- =====

CREATE TRIGGER trg_bloquear_exclusao

BEFORE DELETE ON livro

FOR EACH ROW

EXECUTE FUNCTION bloquear_exclusao();

-- =====

-- TESTE

-- =====

DELETE FROM livro WHERE id_livro = 1; -- temos 5 livros

INSERT INTO livro VALUES (7,'O Alienista',1882,2,2,112,0); -- quantidade 0

DELETE FROM livro WHERE id_livro = 7; -- deve apagar o livro "O Alienista"

-- =====

-- TÉRMINO ATIVIDADE 16

Statistics 1 x

Name	Value
Updated Rows	1
Execute time	0.004s
Start time	Sun Jun 07 15:05:30 BRT 2026
Finish time	Sun Jun 07 15:05:30 BRT 2026
Query	INSERT INTO livro VALUES (7,'O Alienista',1882,2,2,112,0)

General

BRT en WritableSmart Insert56 : 74 : 1556Sel: 0 | 0

47%CPU77%MEM

87%15:07

FileEditNavigateSearchSQL EditorDatabaseWindowHelp

Connections

Filter connections by name

emprestimo24K

emprestimo_livro24K

livro24K

Columns

123 id_livro (int4)

AZ titulo (varchar(150))

123 ano_publicacao (int4)

123 id_autor (int4)

123 id_editora (int4)

123 num_paginas (int4)

123 quantidade (int4)

Constraints

Foreign Keys

Indexes

Dependencies

References

Partitions

Triggers

trg_bloquear_exclusao

Rules

Policies

Foreign Tables

Views

Materialized Views

Indexes

Functions

apagar_autor(int4)

atualizar_pg_livro(int4, int4)

info_autor(int4)

inserir_livro(varchar, int4, int4, int4, int4)

inserir_livro_com_verificacao(varchar, int4, int4)

f media_paginas_autor(int4)

Sequences

Data types

Aggregate functions

Event Triggers

Extensions

Storage

*<postgres> schema.sql

livro

PropertiesDataDiagram

Show SQLEnter a SQL expression to filter results (use Ctrl+Space)

Grid

Text

Record

	123 id_livro	AZ titulo	123 ano_publicacao	123 id_autor	123 id_editora	123 num_paginas	123 quantidade
1	2	Dom Casmurro	1899	2	2	208	5
2	3	A Hora da Estrela	1977	3	3	88	5
3	4	O Hobbit	1937	1	1	336	5
4	5	Laços de Família	1960	3	2	136	5
5	1	O Senhor dos Anéis	1954	1	1	1570	5
6	6	The Book of Lost Tales	1969	1	2	297	5
7	7	O Alienista	1882	2	2	112	0

Refresh

Save

Cancel

Export data

200

7

7 row(s) fetched - 0s, on 2026-06-07 at 15:07:39

General

postgresbibliotecapubliclivro

BRTen

FileEditNavigateSearchSQL EditorDatabaseWindowHelp

SQLCommitRollbackAutopostgrespublic@biblioteca

Search

Connections

Filter connections by name

emprestimo24K

emprestimo_livro24K

livro24K

Columns

123 id_livro (int4)

A-Z titulo (varchar(150))

123 ano_publicacao (int4)

123 id_autor (int4)

123 id_editora (int4)

123 num_paginas (int4)

123 quantidade (int4)

Constraints

Foreign Keys

Indexes

Dependencies

References

Partitions

Triggers

trg_bloquear_exclusao

Rules

Policies

Foreign Tables

Views

Materialized Views

Indexes

Functions

apagar_autor(int4)

atualizar_pg_livro(int4, int4)

info_autor(int4)

inserir_livro(varchar, int4, int4, int4, int4)

inserir_livro_com_verificacao(varchar, int4, int4, int4, int4)

f media_paginas_autor(int4)

Sequences

Data types

Aggregate functions

Event Triggers

Extensions

Storage

*<postgres> schema.sql x livro

-- RAISE EXCEPTION

-- para exibir mensagem apropriada.

-- b)

-- Uma TRIGGER chamada:

-- - trg_bloquear_exclusao

-- executada BEFORE DELETE na tabela:

-- livro

-- =====

CREATE OR REPLACE

FUNCTION

bloquear_exclusao()

RETURNS TRIGGER

AS \$\$

BEGIN

IF OLD.quantidade >0 THEN

RAISE EXCEPTION 'Não podemos excluir registros enquanto há livros existentes!';

END IF;

RETURN OLD;

END;

\$\$ LANGUAGE plpgsql;

-- =====

-- Trigger

-- =====

CREATE TRIGGER trg_bloquear_exclusao

BEFORE DELETE ON livro

FOR EACH ROW

EXECUTE FUNCTION bloquear_exclusao();

-- =====

-- TESTE

-- =====

DELETE FROM livro WHERE id_livro = 1; -- temos 5 livros

INSERT INTO livro VALUES (7, '0 Alienista', 1882, 2, 2, 112, 0); -- quantidade 0

DELETE FROM livro WHERE id_livro = 7; -- deve apagar o livro "0 Alienista"

-- =====

Statistics 1 x

Name	Value
Updated Rows	1
Execute time	0.004s
Start time	Sun Jun 07 15:12:21 BRT 2026
Finish time	Sun Jun 07 15:12:21 BRT 2026
Query	-- quantidade 0 DELETE FROM livro WHERE id_livro = 7

General

BRTenWritableSmart Insert57 : 75 : 1570Sel: 0 | 0

16% CPU78% MEM

87% 15:12

FileEditNavigateSearchSQL EditorDatabaseWindowHelp

SQLCommitRollbackAutopostgrespublic@biblioteca

Search

Connections x

Filter connections by name

bdteste

biblioteca

Schemas

public

Tables

aluno24K

autor24K

editora24K

emprestimo24K

emprestimo_livro24K

livro24K

log_exclusao_livro8K

log_livro8K

Foreign Tables

Views

Materialized Views

Indexes

Functions

log_exclusao_livro()

media_paginas_autor(int4)

Sequences

Data types

Aggregate functions

Event Triggers

Extensions

Storage

System Info

Roles

clima_alerta

fabricao

postgres

queimadas_db

*<postgres> schema.sql xlog_exclusao_livrolivro

-- =====

-- Exercício 2 – Registro Automático de Exclusões

-- =====

-- A biblioteca deseja manter histórico de livros removidos do sistema.

-- Desenvolva:

-- a)Uma FUNCTION chamada:

-- log_exclusao_livro()

-- que registre:

-- -título do livro removido;

-- -data e hora da exclusão;

-- -mensagem informativa no log.

-- b)Uma TRIGGER chamada:

-- trg_log_exclusao

-- executada AFTER DELETE na tabela:

-- livro

-- =====

CREATE OR REPLACE

FUNCTION

log_exclusao_livro()

RETURNS TRIGGER

AS \$\$

BEGIN

INSERT

INTO

log_exclusao_livro(

titulo,

data_log,

mensagem)

VALUES(

OLD.titulo,

CURRENT_TIMESTAMP,

'Registro removido.');

RETURN OLD;

END

\$\$ LANGUAGE plpgsql;

-- =====

Statistics 1 x

Name	Value
Updated Rows	0
Execute time	0.007s
Start time	Sun Jun 07 15:38:56 BRT 2026
Finish time	Sun Jun 07 15:38:56 BRT 2026
Query	-- =====
	-- Exercício 2 – Registro Automático de Exclusões
	-- =====
	-- A biblioteca deseja manter histórico de livros removidos do sistema.
	-- Desenvolva:

General

BRTenWritableSmart Insert116 : 21 : 2870Sel: 0 | 0

14% CPU70% MEM

87% 15:39

FileEditNavigateSearchSQL EditorDatabaseWindowHelp

SQLCommitRollbackAutopostgrespublic@biblioteca

Search

Connections

Filter connections by name

bdteste

biblioteca

Schemas

public

Tables

aluno24K

autor24K

editora24K

emprestimo24K

emprestimo_livro24K

livro24K

log_livro8K

Foreign Tables

Views

Materialized Views

Indexes

Functions

apagar_autor(int4)

atualizar_pg_livro(int4, int4)

f bloquear_exclusao()

info_autor(int4)

inserir_livro(varchar, int4, int4, int4, int4)

inserir_livro_com_verificacao(varchar, int4, int4)

f media_paginas_autor(int4)

Sequences

Data types

Aggregate functions

Event Triggers

Extensions

Storage

System Info

Roles

clima_alerta

fabricio

postgres

queimadas_db

sistema_bancario

Administer

FUNCTIONlog_exclusao_livro()RETURNS TRIGGERAS \$\$BEGININSERTINTOlog_exclusao_livro(titulo,data_log,mensagem)VALUES(OLD.titulo,CURRENT_TIMESTAMP,'Registro removido.');

RETURN OLD;

END

\$\$ LANGUAGE plpgsql;

-- =====

-- Trigger

-- =====

CREATE TRIGGER trg_log_exclusao_livroAFTERDELETEONlivroFOR EACH ROWEXECUTE FUNCTION log_exclusao_livro();

-- =====

-- TESTE

-- =====

DELETE FROM livro WHERE id_livro = 1; -- temos 5 livros

INSERT INTO livro VALUES (8,'0 Alienista',1882,2,2,112,0); -- quantidade 0

DELETE FROM livro WHERE id livro = 8; -- deve apagar o livro "0 Alienista"

Statistics 1

Name	Value
Updated Rows	0
Execute time	0.002s
Start time	Sun Jun 07 15:30:32 BRT 2026
Finish time	Sun Jun 07 15:30:32 BRT 2026
Query	-- ===== -- Trigger -- ===== CREATE TRIGGER trg_log_exclusao_livro AFTER DELETE

General

BRTenWritableSmart Insert126 : 39 : 3043Sel: 0 | 0

13% CPU 80% MEM

87% 15:30

FileEditNavigateSearchSQL EditorDatabaseWindowHelp

SQLCommitRollbackAutopostgrespublic@bibliotecalog_exclusao_livro

Connections

Filter connections by name

bdteste

biblioteca

Schemas

public

Tables

aluno24K

autor24K

editora24K

emprestimo24K

emprestimo_livro24K

livro24K

log_exclusao_livro8K

log_livro8K

Foreign Tables

Views

Materialized Views

Indexes

Functions

apagar_autor(int4)

atualizar_pg_livro(int4, int4)

f bloquear_exclusao()

info_autor(int4)

inserir_livro(varchar, int4, int4, int4, int4)

inserir_livro_com_verificacao(varchar, int4, int4)

f log_exclusao_livro()

f media_paginas_autor(int4)

Sequences

Data types

Aggregate functions

Event Triggers

Extensions

Storage

System Info

Roles

clima_alerta

fabricao

postgres

queimadas_db

FUNCTION

log_exclusao_livro()

RETURNS TRIGGER

AS \$\$

BEGIN

INSERT

INTO

log_exclusao_livro(

titulo,

data_log,

mensagem)

VALUES(

OLD.titulo,

CURRENT_TIMESTAMP,

'Registro removido.');

RETURN OLD;

END

\$\$ LANGUAGE plpgsql;

-- =====

-- Trigger

-- =====

CREATE TRIGGER trg_log_exclusao_livro

AFTER

DELETE

ON

livro

FOR EACH ROW

EXECUTE FUNCTION log_exclusao_livro();

-- =====

-- TESTE

-- =====

DELETE FROM livro WHERE id_livro = 1; -- temos 5 livros

INSERT INTO livro VALUES (8,'O Alienista',1882,2,2,112,0); -- quantidade 0

DELETE FROM livro WHERE id livro = 8; -- deve apagar o livro "O Alienista"

Statistics 1

Name	Value
Updated Rows	1
Execute time	0.002s
Start time	Sun Jun 07 15:31:45 BRT 2026
Finish time	Sun Jun 07 15:31:45 BRT 2026
Query	-- temos 5 livros INSERT INTO livro VALUES (8,'O Alienista',1882,2,2,112,0)

General

BRTenWritableSmart Insert131 ...3230

43°C CPU 68%

MEM

87% 15:32

FileEditNavigateSearchSQL EditorDatabaseWindowHelp

SQLCommitRollbackAutopostgrespublic@biblioteca

Connections

Filter connections by name

bdteste

biblioteca

Schemas

public

Tables

aluno24K

autor24K

editora24K

emprestimo24K

emprestimo_livro24K

livro24K

log_exclusao_livro8K

log_livro8K

Foreign Tables

Views

Materialized Views

Indexes

Functions

apagar_autor(int4)

atualizar_pg_livro(int4, int4)

bloquear_exclusao()

info_autor(int4)

inserir_livro(varchar, int4, int4, int4, int4)

inserir_livro_com_verificacao(varchar, int4, int4)

log_exclusao_livro()

media_paginas_autor(int4)

Sequences

Data types

Aggregate functions

Event Triggers

Extensions

Storage

System Info

Roles

clima_alerta

fabricao

postgres

queimadas_db

*<postgres> schema.sql xlog_exclusao_livrolivro

log_exclusao_livro()
RETURNS TRIGGER
AS \$\$
BEGIN
 INSERT
 INTO
 log_exclusao_livro(
 titulo,
 data_log,
 mensagem)
VALUES(
 OLD.titulo,
 CURRENT_TIMESTAMP,
 'Registro removido.');

RETURN OLD;
END
\$\$ LANGUAGE plpgsql;

-- =====
-- Trigger
-- =====
CREATE TRIGGER trg_log_exclusao_livro
AFTER
DELETE
 ON
 livro
FOR EACH ROW
EXECUTE FUNCTION log_exclusao_livro();

-- =====
-- TESTE
-- =====
DELETE FROM livro WHERE id_livro = 1; -- temos 5 livros
INSERT INTO livro VALUES (8, '0 Alienista', 1882, 2, 2, 112, 0); -- quantidade 0
DELETE FROM livro WHERE id_livro = 8; -- deve apagar o livro e registrar no log

Statistics 1 x

Name	Value
Updated Rows	1
Execute time	0.002s
Start time	Sun Jun 07 15:33:05 BRT 2026
Finish time	Sun Jun 07 15:33:05 BRT 2026
Query	-- quantidade 0 DELETE FROM livro WHERE id_livro = 8

General

BRT en WritableSmart Insert132 : 80 : 3310

42°C CPU 34% MEM87% 15:33

DBever 26.1.0 - log_exclusao_livro

FileEditNavigateSearchSQL EditorDatabaseWindowHelp

SQLCommitRollbackAutopostgrespublic@biblioteca

log_exclusao_livrolivro

Connections

Filter connections by name

bdteste

biblioteca

Schemas

public

Tables

aluno24K

autor24K

editora24K

emprestimo24K

emprestimo_livro24K

livro24K

log_exclusao_livro8K

log_livro8K

Foreign Tables

Views

Materialized Views

Indexes

Functions

apagar_autor(int4)

atualizar_pg_livro(int4, int4)

f bloquear_exclusao()

info_autor(int4)

inserir_livro(varchar, int4, int4, int4, int4)

inserir_livro_com_verificacao(varchar, int4, int4)

f log_exclusao_livro()

f media_paginas_autor(int4)

Sequences

Data types

Aggregate functions

Event Triggers

Extensions

Storage

System Info

Roles

clima_alerta

fabricio

postgres

queimadas_db

PropertiesDataDiagram

Show SQLEnter a SQL expression to filter results (use Ctrl+Space)

Grid

123id_log

A-Z titulo

data_log

A-Z mensagem

1

1

O Alienista

06-07 15:33:05.765

Registro removido.

Text

Record

RefreshSaveCancelExport data2001

Generalpostgresbibliotecapubliclog_exclusao_livroBRTen

43%CPU71%MEM87%15:33

FileEditNavigateSearchSQL EditorDatabaseWindowHelp

CommitRollbackAuto

postgrespublic@biblioteca

Connections x

Filter connections by name

postgreslocalhost:5432

Databases

- Aula6
- Aula7
- PROVA_3
- aula12
- bdaula
- bdaula_arley
- bdteste

biblioteca

- Schemas
 - public
 - Tables
 - aluno24K
 - autor24K
 - editora24K
 - emprestimo24K
 - emprestimo_livro24K
 - livro24K
 - Columns
 - Constraints
 - Foreign Keys
 - Indexes
 - Dependencies
 - References
 - Partitions
 - Triggers
 - trg_log_exclusao_livro
 - Rules
 - Policies
 - log_exclusao_livro24K
 - log_livro8K
 - Foreign Tables
 - Views
 - Materialized Views
 - Indexes
 - Functions
 - Sequences
 - Data types

*<postgres> schema.sql x livro

-- =====
-- Exercício 3 – Controle de Aumento Excessivo de Estoque
-- =====
-- Durante auditorias, foi identificado que alguns operadores cadastraram quanti-
-- dades irreais de livros.
-- Crie:
-- a) Uma FUNCTION chamada:
-- validar_limite_estoque() que:
-- -impeça UPDATE quando:
-- -quantidade > 100
-- -apresente mensagem de erro usando:
-- RAISE EXCEPTION
-- b) Uma TRIGGER chamada:
-- trg_validar_limite
-- executada BEFORE UPDATE na tabela livro
-- =====

CREATE OR REPLACE
FUNCTION
validar_limite_estoque()
RETURNS TRIGGER
AS \$\$
BEGIN
 IF NEW.quantidade >100 THEN
 RAISE EXCEPTION 'Solicite autorização para inserir mais de 100 quantidades.';
 END IF;

RETURN NEW;
END;

\$\$ LANGUAGE plpgsql;

Statistics 1 x

Name	Value
Updated Rows	0
Execute time	0.003s
Start time	Sun Jun 07 16:02:32 BRT 2026
Finish time	Sun Jun 07 16:02:32 BRT 2026
Query	-- ===== -- Exercício 3 – Controle de Aumento Excessivo de Estoque -- ===== -- Durante auditorias, foi identificado que alguns operadores cadastraram quanti- -- dades irreais de livros. -- Crie: -- a) Uma FUNCTION chamada: -- validar_limite_estoque() que: -- -impeça UPDATE quando: -- -quantidade > 100

General

BRT en Writable Smart Insert 163 : 21 : 4240

46°CPU43% MEM

87% 16:02

FileEditNavigateSearchSQL EditorDatabaseWindowHelp

Connections x

Filter connections by name

postgres localhost:5432

Databases

- Aula6
- Aula7
- PROVA_3
- aula12
- bdaula
- bdaula_arley
- bdteste

biblioteca

- Schemas
 - public
 - Tables
 - aluno24K
 - autor24K
 - editora24K
 - emprestimo24K
 - emprestimo_livro24K
 - livro24K
 - Columns
 - Constraints
 - Foreign Keys
 - Indexes
 - Dependencies
 - References
 - Partitions
 - Triggers
 - trg_log_exclusao_livro
 - Rules
 - Policies
 - log_exclusao_livro24K
 - log_livro8K
 - Foreign Tables
 - Views
 - Materialized Views
 - Indexes
 - Functions
 - Sequences
 - Data types

*<postgres> schema.sql x livro

```
-- RAISE EXCEPTION
-- b)Uma TRIGGER chamada:
--   trg_validar_limite
--   executada BEFORE UPDATE na tabela livro
-- =====
CREATE OR REPLACE
FUNCTION
validar_limite_estoque()
RETURNS TRIGGER
AS $$
BEGIN
    IF NEW.quantidade >100 THEN
        RAISE EXCEPTION 'Solicite autorização para inserir mais de 100 quantidades.';
    END IF;

    RETURN NEW;
END;

$$ LANGUAGE plpgsql;
-- =====
-- Trigger
-- =====
CREATE TRIGGER trg_validar_limite_estoque
BEFORE
UPDATE
ON
    livro
FOR EACH ROW
EXECUTE FUNCTION validar_limite_estoque();
-- =====
```

Statistics 1 x

Name	Value
Updated Rows	0
Execute time	0.001s
Start time	Sun Jun 07 16:03:02 BRT 2026
Finish time	Sun Jun 07 16:03:02 BRT 2026
Query	-- ===== -- Trigger -- ===== CREATE TRIGGER trg_validar_limite_estoque BEFORE UPDATE ON livro FOR EACH ROW EXECUTE FUNCTION validar_limite_estoque()

General

BRT en Writable Smart Insert 173 : 43 : 4423

46°CPU20%

MEM

87% 16:03

FileEditNavigateSearchSQL EditorDatabaseWindowHelp

CommitRollbackAuto

postgrespublic@biblioteca

Connections

Filter connections by name

postgres localhost:5432

Databases

- Aula6
- Aula7
- PROVA_3
- aula12
- bdaula
- bdaula_arley
- bdteste
- biblioteca
 - Schemas
 - public
 - Tables
 - aluno24K
 - autor24K
 - editora24K
 - emprestimo24K
 - emprestimo_livro24K
 - livro24K
 - Columns
 - Constraints
 - Foreign Keys
 - Indexes
 - Dependencies
 - References
 - Partitions
 - Triggers
 - trg_log_exclusao_livro
 - Rules
 - Policies
 - log_exclusao_livro24K
 - log_livro8K
 - Foreign Tables
 - Views
 - Materialized Views
 - Indexes
 - Functions
 - Sequences
 - Data types

*<postgres> schema.sql x livro

```
-- =====
CREATE OR REPLACE
FUNCTION
validar_limite_estoque()
RETURNS TRIGGER
AS $$
BEGIN
    IF NEW.quantidade >100 THEN
        RAISE EXCEPTION 'Solicite autorização para inserir mais de 100 quantidades.';
    END IF;

    RETURN NEW;
END;

$$ LANGUAGE plpgsql;
-- =====
-- Trigger
-- =====
CREATE TRIGGER trg_validar_limite_estoque
BEFORE
UPDATE
ON
    livro
FOR EACH ROW
EXECUTE FUNCTION validar_limite_estoque();
-- =====
-- TESTE
-- =====
UPDATE livro SET quantidade = 100 WHERE id_livro = 1; --aumenta de 5 para 100!
UPDATE livro SET quantidade = 101 WHERE id_livro = 1; --impede a atualização.
```

Statistics 1 x

Name	Value
Updated Rows	1
Execute time	0.003s
Start time	Sun Jun 07 16:03:28 BRT 2026
Finish time	Sun Jun 07 16:03:28 BRT 2026
Query	-- ===== -- TESTE -- ===== UPDATE livro SET quantidade = 100 WHERE id_livro = 1

General

BRT en Writable Smart Insert 177 : 79 : 4557

45°C CPU 43%

MEM

87% 16:03

FileEditNavigateSearchSQL EditorDatabaseWindowHelp

Connections x

Filter connections by name

postgres localhost:5432

Databases

- Aula6
- Aula7
- PROVA_3
- aula12
- bdaula
- bdaula_arley
- bdteste

biblioteca

- Schemas
 - public
 - Tables
 - aluno24K
 - autor24K
 - editora24K
 - emprestimo24K
 - emprestimo_livro24K
 - livro24K
 - Columns
 - Constraints
 - Foreign Keys
 - Indexes
 - Dependencies
 - References
 - Partitions
 - Triggers
 - trg_log_exclusao_livro
 - Rules
 - Policies
 - log_exclusao_livro24K
 - log_livro8K
 - Foreign Tables
 - Views
 - Materialized Views
 - Indexes
 - Functions
 - Sequences
 - Data types

*<postgres> schema.sql

livro x

PropertiesDataDiagram

Show SQL | Enter a SQL expression to filter results (use Ctrl+Space)

	id_livro	titulo	ano_publicacao	id_autor	id_editora	num_paginas	quantidade
1	2	Dom Casmurro	1899	2	2	208	5
2	3	A Hora da Estrela	1977	3	3	88	5
3	4	O Hobbit	1937	1	1	336	5
4	5	Laços de Família	1960	3	2	136	5
5	6	The Book of Lost Tales	1969	1	2	297	5
6	1	O Senhor dos Anéis	1954	1	1	1570	100

Refresh

SaveCancel

Export data

200

6

6 row(s) fetched - 0.001s, on 2026-06-07 at 16:03:45

General

postgresbibotecapubliclivro

BRTen

44°CPU29%

MEM

87%16:03

Connections x

Filter connections by name

postgres localhost:5432

Databases

Aula6

Aula7

PROVA_3

aula12

bdaula

bdaula_arley

bdteste

biblioteca

Schemas

public

Tables

aluno 24K

autor 24K

editora 24K

emprestimo 24K

emprestimo_livro 24K

livro 24K

Columns

Constraints

Foreign Keys

Indexes

Dependencies

References

Partitions

Triggers

trg_log_exclusao_livro

Rules

Policies

log_exclusao_livro 24K

log_livro 8K

Foreign Tables

Views

Materialized Views

Indexes

Functions

Sequences

Data types

```
-- =====
CREATE OR REPLACE
FUNCTION
validar_limite_estoque()
RETURNS TRIGGER
AS $$
BEGIN
    IF NEW.quantidade >100 THEN
        RAISE EXCEPTION 'Solicite autorização para inserir mais de 100 quantidades.';
    END IF;

    RETURN NEW;
END;

$$ LANGUAGE plpgsql;
-- =====
-- Trigger
-- =====
CREATE TRIGGER trg_validar_limite_estoque
BEFORE
UPDATE
ON
    livro
FOR EACH ROW
EXECUTE FUNCTION validar_limite_estoque();
-- =====
-- TESTE
-- =====
UPDATE livro SET quantidade = 100 WHERE id_livro = 1; --aumenta de 5 para 100!
UPDATE livro SET quantidade = 101 WHERE id_livro = 1; --impede a atualização.
```

Statistics 1 x

SQL Error [P0001]: ERROR: Solicite autorização para inserir mais de 100 quantidades.
Where: PL/pgSQL function validar_limite_estoque() line 4 at RAISE

Go to Error

Show log

Details >>

```
--aumenta de 5 para 100!  
UPDATE livro SET quantidade = 101 WHERE id_livro = 1
```